

PopSystem Dev Guide v2.0 (Clean Build)

Master Development Plan - NewPOPSys v1.38

Status: Final **Author:** System Architect **Date:** 2026-01-02 **Source:** SOW v1.38 & SRS v1.38

1. Executive Summary

NewPOPSys v1 is a pilot-grade platform (MPV) designed to manage Point-of-Purchase (POP) installation and verification for retail chains.

- **Goal:** Replace legacy "PrecisionPOP" system with a scalable, modern architecture.
- **Budget:** \$150,000 (Fixed Bid).
- **Timeline:** 13 Weeks (Target Beta: End Q1 2026).
- **Key Deliverables:**
 1. **Mobile PWA:** Field Rep execution (Offline-capable).
 2. **Brand Portal:** Campaign management dashboard.
 3. **PSP Portal:** Print & logistics operations.
 4. **Store Portal:** Store manager visibility.

2. Technology Stack (Authoritative)

All development must adhere strictly to this stack per SUPP-012.

Layer	Technology	Decision	Purpose
Frontend	Next.js 14+	App Router	PWA Scaffold, Web Portals, SSR.
Language	TypeScript	5.3+	Strict typing across full stack.
Styling	Tailwind CSS	3.x	Utility-first styling.
State	Context API	-	Client state (No external libs).
Backend	Fastify	4.x	High-performance API Service.
Database	PostgreSQL	16 (RDS)	Primary System of Record.
ORM	Drizzle	Latest	SQL-like, type-safe queries.
Async	Redis + BullMQ	7.x	Webhooks, Exports, Job Queues.
Build	Turborepo	Latest	Monorepo orchestration.

Infra AWS ECS/Fargate Containerized hosting.

Explicit Exclusions:

- NO **Vite** (Next.js handles build).
- NO **Zustand** (Context API sufficient).
- NO **TanStack Query** (Use standard fetch/SWR pattern unless mandated).
- NO **Supabase** (Custom Auth + RDS).

3. Architecture Overview

3.1 Monorepo Structure (Turborepo)

```
apps/
├── mobile-pwa/      # Next.js (Field Reps)
├── brand-portal/    # Next.js (Brand Admins)
├── psp-portal/      # Next.js (Print/Ops)
├── store-portal/    # Next.js (Store Mgrs)
└── api-server/     # Fastify (Core logic)
packages/
├── ui/              # Shared React components
├── database/        # Drizzle Schema & Migrations
└── types/           # Shared Zod schemas/TS types
```

3.2 Key Flows

1. **Campaign Management:** Brands define Campaigns (JSONB config in Postgres).
2. **Logistics:** PSPs manage print queues and shipments (Integrated via Webhooks/Redis).
3. **Execution:** Field Reps capture photos (PWA Offline Drafts -> Sync to Fastify).
4. **Validation:** AI/Manual review of photos (Stored in S3).

4. Development Roadmap

Sprint	Theme	Key Deliverables
S0	Foundation	Monorepo, Fastify Setup, Drizzle Schema, CI/CD.
S1	Auth & Core	JWT Auth, RBAC, Basic UI Shell (Next.js).
S2	Mobile Core	Field App PWA, Photo Capture, Offline Drafts.
S3	Brand Portal	Campaign Builder, Store Selection.
S4	PSP Ops	Order Queue, Shipping Integrations (BullMQ).
S5	Integration	Webhooks, Store Portal, Reporting.
S6	Launch	E2E Testing, Load Tests, Production Deploy.

5. Budget & Resources

- **Total Budget:** \$150,000
 - Development: \$100,000
 - Design/QA: \$30,000
 - Infrastructure/Ops: \$20,000
 - **Team:** 2 Full Stack Devs, 1 QA/Lead.
-

Last Updated: 2026-01-02

Sprint Roadmap

Sprint	Theme	Dates (Est)	Key Goals
S0	Foundation	Jan 6 - Jan 17	Monorepo, CI/CD, Core Schemas, Fastify/Next.js Init
S1	Auth & Core UI	Jan 20 - Jan 31	Auth Service (Fastify), UI Shells (Next.js)
S2	Mobile Core	Feb 3 - Feb 14	Field PWA, Photo Capture, Offline Sync
S3	Brand Portal	Feb 17 - Feb 28	Campaign Management, Targeting
S4	PSP Ops	Mar 3 - Mar 14	Fulfillment, Print Queues, Shipping (BullMQ)
S5	Integration	Mar 17 - Mar 28	Store Portal, Webhooks, Reporting
S6	Launch	Mar 31 - Apr 4	E2E Testing, Production Deploy, Handoff

Sprint S0: Foundation

Setup Tasks

- ☐ **S0-01:** Initialize Turborepo (pnpm).
- ☐ **S0-02:** Create apps/api-server (Fastify + TypeScript).
- ☐ **S0-03:** Create apps/mobile-pwa, apps/brand, apps/psp, apps/store (Next.js 14).
- ☐ **S0-04:** Configure Shared packages/ui (Tailwind).
- ☐ **S0-05:** Configure packages/database (Drizzle + Postgres connection).

Infrastructure

- ☐ **S0-06:** Set up Redis (Local + Cloud).
- ☐ **S0-07:** Configure BullMQ for job queues.
- ☐ **S0-08:** Setup CI/CD (GitHub Actions).

Validation

- ☐ **Tests:** CI passes lint/build.
- ☐ **Run:** All 5 apps start locally.

Sprint S1: Authentication & Core UI

Auth Service (Fastify)

- ☐ **S1-01:** Implement Login/Register Endpoints.
- ☐ **S1-02:** JWT Generation & Verification Middleware.
- ☐ **S1-03:** Password Hashing (Argon2).

Frontend Integration (Next.js)

- ☐ **S1-04:** Build Login Page (Shared Component).
- ☐ **S1-05:** Implement Auth Context (Client Side Session).
- ☐ **S1-06:** Role-Based Route Protection (Middleware).

Core UI

- ☐ **S1-07:** Implement App Shell (Sidebar/Header) for Portals.

Sprint S2: Mobile Core (PWA)

PWA Scaffold (Next.js)

- ☐ **S2-01:** Configure Manifest & Service Workers (next-pwa).
- ☐ **S2-02:** Offline Fallback Pages.

Core Features

- ☐ **S2-03:** Campaign List (Fetch from API).
- ☐ **S2-04:** Task Execution Flow.
- ☐ **S2-05:** Photo Capture Interface (HTML5 Camera API).
- ☐ **S2-06:** Offline Draft Storage (IndexedDB wrapper).

Sync

- ☐ **S2-07:** Background Sync Logic (Upload Drafts when Online).

Sprint S3: Brand Portal

Campaigns

- ☐ **S3-01:** Campaign Builder UI (Multi-step form).
- ☐ **S3-02:** Targeting Logic (Select Stores via API).
- ☐ **S3-03:** Campaign Dashboard & Metrics.

Assets

- ☐ **S3-04:** POP Kit Configuration (JSON Editor).
- ☐ **S3-05:** Asset Upload (S3 Direct Upload).

Sprint S4: PSP Operations Portal

Order Management

- ☐ **S4-01:** Incoming Campaign Queue.
- ☐ **S4-02:** Print Job Grouping.
- ☐ **S4-03:** Shipping Label Generation.

Integrations (BullMQ)

- ☐ **S4-04:** Async Job for Shipping Label API (FedEx/UPS).
- ☐ **S4-05:** Order Status Webhooks.

Sprint S5: Store Portal & Integration

Store Portal

- ☐ **S5-01:** Store Dashboard (Compliance Score).
- ☐ **S5-02:** Issue Reporting.

System Integration

- ☐ **S5-03:** Webhook Outbox Implementation (Redis).
- ☐ **S5-04:** Data Export Jobs (BullMQ).
- ☐ **S5-05:** Global Reporting Dashboard.

Sprint S6: Launch Prep

QA & Testing

- ☐ **S6-01:** E2E Testing Suite (Playwright key flows).
- ☐ **S6-02:** Load Testing (K6 vs Fastify API).

Production

- ☐ **S6-03:** Production Env Config (AWS).
- ☐ **S6-04:** Final Security Audit.
- ☐ **S6-05:** Pilot Launch.

Technology Stack Decisions (ADRs)

Status: Authoritative **Source:** SUPP-012

ADR-001: Next.js (App Router)

- **Decision:** Use Next.js 14+ for all frontend applications (Mobile PWA, Brand, PSP, Store).
- **Rationale:**
 - Unified framework for SSR and Static generation.
 - "PWA Scaffold" capability explicitly requested in SRS.
 - Simplifies deployment via container or Vercel-like adapters.
- **Consequences:** Strict adherence to App Router patterns (app/ directory).

ADR-002: Fastify (Backend)

- **Decision:** Use Fastify 4.x for the dedicated API Service.
- **Rationale:**
 - High performance (required for high-volume photo uploads).
 - Schema-based validation (Zod/JSON Schema) native support.
- **Consequences:** Separate apps/api-server deployment from Next.js frontends.

ADR-003: Drizzle ORM

- **Decision:** Use Drizzle ORM with PostgreSQL.
- **Rationale:**
 - Type-safety with low runtime overhead.
 - Control over SQL queries (critical for complex Campaign targeting logic).
 - JSONB support for Survey Schemas.

ADR-004: Redis + BullMQ

- **Decision:** Use Redis 7.x and BullMQ for async operations.
- **Rationale:** SRS Mandates "Outbox pattern" and "Reliable Webhooks".
- **Use Cases:**
 - Webhook delivery retries.
 - Large Export generation (PDF/CSV).
 - Photo Processing/AI Validation jobs.

ADR-005: Turborepo

- **Decision:** Use Turborepo for Monorepo management.
- **Rationale:**
 - Efficient caching of builds.
 - Dependency management for shared packages (ui, database).

Verified against SRS v1.38

AI Development Specifications

Guidance for AI Agents: Use this context for all code generation.

1. Project Context

- **Project:** NewPOPSys v1 (Point of Purchase Management).
- **Type:** Monorepo (Turborepo).
- **Strict Stack:** Next.js, Fastify, Drizzle, Redis.

2. Code Patterns

2.1 Frontend (Next.js)

- **Directory Structure:** Use App Router (app/page.tsx, app/layout.tsx).

- **Data Fetching:** Use `fetch` in Server Components where possible. Use Client Components only for interactivity.
- **State:** Use `React.useState` OR `React.useContext`. **Do NOT install Redux/Zustand.**
- **Styling:** Tailwind CSS utility classes.

2.2 Backend (Fastify)

- **Route Definition:**

```
fastify.post('/api/campaigns', {  
  schema: { body: CampaignSchema },  
  handler: async (req, reply) => { ... }  
})
```

- **Async Jobs (BullMQ):**

```
await campaignQueue.add('process-photos', { campaignId }, { attempts:
```

2.3 Database (Drizzle)

- **Schema:** Define in `packages/database/src/schema/*.ts`.
- **Query:**

```
const result = await db.select().from(campaigns).where(eq(campaigns.id
```

3. Automation Scripts

- **Context Loading:** Run `npm run ai:context` (if available) to concatenate SUPP files.
- **Verification:** Always run `npm run lint` and `npm run typecheck` before committing.

No Vite. No Supabase.

Testing Strategy

1. Unit Testing (Vitest)

- **Scope:** Utility functions, Hooks, API Handlers.
- **Tool:** **Vitest** (Fast, native ESM).
- **Target:** 80% Coverage on core logic.

2. Integration Testing (Vitest + Supertest)

- **Scope:** API Endpoints (Fastify).
- **Strategy:** Spin up ephemeral Postgres/Redis containers.

3. E2E Testing (Playwright)

- **Scope:** Critical User Flows (Login, Photo Capture).
- **Tool:** **Playwright**.
- **Target:** Smoke tests for every deploy.

4. Manual Testing

- **Scope:** UX Polish, Edge Cases.